Data Engineering

Linux Notes: Interview Ready

Fundamentals, the commands that matter for data work, one-liners, and 25 likely interview questions.

1. Linux in 60 seconds

- Technically Linux is a **kernel**, not a full OS. Distributions (Ubuntu, Red Hat, Fedora, Amazon Linux, SUSE...) bundle the kernel with tools to make a complete OS.
- Open source, created in **1991**, written mainly in C and Assembly (Rust is now used too). Part of the **Unix-like** family.
- **Why data engineers use it:** most data servers, clusters and pipelines run on Linux. It is secure, multiuser, multitasking, light on resources, stable, and easy to automate with scripts.
- Drawback: steeper learning curve than Windows because most work happens in the terminal.
- **Kernel** = heart of the OS; manages hardware (CPU, memory, disk, devices). **Shell** = the interface where you type commands; it passes them to the kernel. Simplified: OS = Kernel + Shell.
- Package families: **Debian-based** (Ubuntu) use **apt**; **Red Hat-based** (RHEL, CentOS, Fedora, Amazon Linux) use **yum/dnf**. RHEL is a paid product; Amazon Linux behaves very much like it.

```
Layers (outside to inside):
  Apps  ->  Shell + Utilities  ->  Kernel  ->  Hardware
```

Types of users

`Root`	The admin: full control over the system.
`Normal user`	Created by root; limited rights.
`System user`	Created automatically by software (for example, installing a database creates its own service user).
`sudo su -`	Switch to root.
`su - <user>`	Switch to another user.
`whoami / id`	Show current user / user and group IDs.
`~`	Shortcut for your home directory.

2. Files and directories

`pwd`	Print the current directory.
`ls -lah`	List files: long format, hidden files, human-readable sizes.
`cd <dir> / cd .. / cd ~`	Change directory / go up one level / go home.
`. and ..`	Current directory and parent directory.
`mkdir -p a/b/c`	Create nested directories.
`touch f`	Create an empty file (or update its timestamp).
`cp -r src dst`	Copy files or directories.
`mv src dst`	Move, or rename if the destination is a new name.
`rm f / rm -r dir`	Delete a file / a directory tree. There is no recycle bin, so be careful.
`rmdir`	Remove an empty directory.

`find /data -name "*.csv"`	Search the directory tree by name, size, time and more.
`locate name`	Fast filename search using an index.
`tree`	Show directories as a tree (install if missing).
`stat f`	Detailed file info: size, permissions, timestamps.

3. Viewing and editing files

`cat f`	Print a file. **cat a b > c** joins files.
`tac f`	Print a file in reverse line order.
`less f / more f`	Scroll through big files page by page (prefer less).
`head -n 5 f`	First 5 lines (peek at a CSV header).
`tail -n 20 f`	Last 20 lines.
`tail -f f`	Follow a file as it grows (live logs).
`nano f`	Beginner-friendly terminal editor.
`vi / vim f`	Powerful editor. **i** insert, **Esc** normal mode, **:wq** save and quit, **:q!** quit without saving, **:set number** show line numbers.
`man ls`	Manual page for any command.
`history`	Commands you ran earlier.
`echo "hi"`	Print text.

4. Redirection and pipes

`>`	**Overwrite** a file with the output. *echo "hi" > f1* replaces everything in f1.
`>>`	**Append** output to the end of a file.
`2> err.log`	Redirect errors (stderr) only.
`> out.log 2>&1`	Send both output and errors to one file.
`|`	**Pipe**: feed the output of one command into the next.
`tee f`	Write to a file and still show it on screen.
`cat > f`	Type text, press Enter, then **Ctrl+D** to save and finish.

```
yum list installed | grep python      # chain commands with a pipe
echo "text" | tee file.txt            # write and display
```

Interview tip: Know the difference between > and >> cold. Overwriting a file by mistake is the classic Linux horror story.

5. Text processing: the data engineer toolkit

`grep "err" f`	Search for text. Useful flags: **-i** ignore case, **-r** recursive, **-v** invert, **-c** count, **-n** line numbers.
`sed "s/old/new/g" f`	Find and replace across a file or stream.
`awk "{print $1}" f`	Process columns. **-F,** sets a comma delimiter, so awk -F, "{print $2}" prints column 2 of a CSV.
`cut -d, -f1,3 f`	Extract fields 1 and 3 using a comma delimiter.
`sort f`	Sort lines. **-n** numeric, **-r** reverse, **-u** unique.
`uniq`	Remove *adjacent* duplicate lines (so sort first). **-c** counts.

`wc -l f`	Count lines (**-w** words, **-c** bytes).
`tr "a-z" "A-Z"`	Translate or delete characters.
`paste f1 f2`	Merge files side by side.
`diff a b`	Show differences (prints nothing if identical).
`cmp a b`	Byte-by-byte check whether two files are identical.
`comm a b`	Compare two *sorted* files: unique to each and common lines.

6. Permissions and ownership

ls -l shows e.g. `-rwxr-xr--`: type, then owner / group / others. r=4, w=2, x=1.

`chmod 755 f`	Owner rwx, group and others r-x. **chmod +x script.sh** makes it executable.
`chmod 644 f`	Owner rw-, everyone else read-only (typical data file).
`chown user:group f`	Change owner and group.
`chgrp group f`	Change group only.
`umask 022`	Default for new files: **644** for files and **755** for directories.
`setfacl / getfacl`	Set / view fine-grained access control lists.
`sudo / visudo`	Run as root / edit the sudoers file safely.

7. Processes and jobs

`ps aux`	Snapshot of all running processes.
`top / htop`	Live view of CPU and memory use (htop is friendlier).
`kill <pid>`	Ask a process to stop (SIGTERM). **kill -9** forces it (SIGKILL), as a last resort.
`killall name`	Stop processes by name.
`jobs / bg / fg`	List jobs / resume in background / bring to foreground.
`cmd &`	Run a command in the background.
`nohup cmd &`	Keep running after you log out.
`nice / renice`	Start a process with, or change, its priority.
`uptime`	How long the system has run, plus load average.
`time cmd`	Measure how long a command takes.

8. Disk and memory

`df -h`	Disk space per file system (is the disk full?).
`du -sh dir`	Size of a directory. **du -sh *** finds what is eating space.
`lsblk`	List disks and partitions.
`mount / umount`	Attach / detach a file system. Persistent mounts live in **/etc/fstab**.
`blkid`	Show block device IDs and types.
`fdisk / parted`	Partition a disk.
`mkfs`	Create a file system on a partition.
`fsck`	Check and repair a file system (unmounted).
`free -h`	Memory and swap usage.
`vmstat / iostat`	Memory, CPU and disk I/O statistics.

`lscpu / lshw`	CPU and hardware details.

> **Interview tip: df** reports free space per file system; **du** reports how much a folder uses. Use df to see the disk is full, then du to find the culprit.

9. Archiving and compression

```
tar -czf data.tar.gz /path/dir     # create gzip archive
tar -xzf data.tar.gz               # extract
tar -cf data.tar /path/dir         # plain tarball (no compression)
tar -xf data.tar                   # extract plain tarball
zip -r data.zip dir/    unzip data.zip
gzip f / gunzip f.gz    bzip2 f / bunzip2 f.bz2    xz f / unxz f.xz
zcat f.gz | head        # peek into a gzip file without extracting
```

- Flags: **c** create, **x** extract, **z** gzip, **v** verbose, **f** file name. Speed vs size: gzip is fast, bzip2 smaller, xz smallest but slowest.

10. Networking and data transfer

`ping host`	Check reachability.
`ip addr / ip route`	Show IP addresses / routing table (modern replacement for ifconfig).
`ss -tulpn`	Listening sockets and ports (replaces netstat).
`curl -O url`	Transfer data with URLs; great for calling APIs: **curl -X GET url**.
`wget url`	Download files from the web.
`ssh user@host`	Secure remote login.
`scp f user@host:/path`	Secure copy between machines.
`rsync -avz src/ dst/`	Efficient sync that copies only changes; works over SSH.
`nslookup / dig`	DNS lookups.
`traceroute`	Trace the network path.
`nc host port`	Test whether a port is open.

11. Packages and services

`apt-get update`	Refresh the package list (Debian/Ubuntu).
`apt-get install pkg`	Install. Also **upgrade**, **remove**; **apt-cache search pkg** to find packages.
`yum install pkg`	Install on Red Hat family; also update, remove. **dnf** is the newer version.
`rpm -i / -e`	Install / remove a local .rpm file.
`dpkg -i / -r`	Install / remove a local .deb file.
`systemctl start|stop|restart svc`	Control a service.
`systemctl enable|disable svc`	Start (or not) at boot.
`systemctl status svc`	Is it running? Shows recent log lines.
`service svc start`	Older init-style equivalent.

12. Scheduling with cron

Cron is how most batch jobs (nightly loads, cleanups, report refreshes) get scheduled on a Linux machine.

`crontab -e`	Edit your cron jobs.
`crontab -l`	List them.
`crontab -r`	Remove them all (careful).
`at 09:00`	Run a one-time job at a set time.
`batch`	Run when system load is low.
`sleep 5`	Pause for 5 seconds (handy in scripts).

```
# minute  hour  day-of-month  month  day-of-week   command
  30      2     *             *      *             /home/user/load_data.sh
  0       */6   *             *      1-5           /home/user/refresh.sh >> r.log 2>&1
```

Interview tip: Be able to read a cron line: "30 2 * * *" means every day at 02:30. Cron uses a minimal environment, so use full paths in scripts.

13. Logs and troubleshooting

`tail -f app.log`	Watch a log live.
`grep -i error app.log | tail`	Latest error lines.
`journalctl -u svc -f`	Follow a service log (systemd).
`dmesg`	Kernel and hardware messages.
`lsof file`	Which process has this file open (also lsof -i :8080 for a port).
`strace cmd`	Trace system calls of a command.
`watch -n 1 free -h`	Re-run a command every second.
`uname -a`	Kernel and system info.
`hostname`	Show or set the hostname.
`who / w / last`	Who is logged in and what they are doing.

14. Real data-engineering one-liners

```
head -n 1 sales.csv                          # view the CSV header
wc -l sales.csv                              # row count (minus 1 for header)
cut -d, -f2 sales.csv | sort | uniq -c       # frequency count of column 2
awk -F, '{sum+=$3} END {print sum}' sales.csv    # total of column 3
sort -u ids.txt > unique_ids.txt             # de-duplicate a list
grep -c "ERROR" pipeline.log                 # count errors
grep "2026-11-0" pipeline.log | grep ERROR   # errors for given days
sed -i 's/NULL//g' data.csv                  # clean values in place
split -l 1000000 big.csv part_               # split a huge file
zcat logs.gz | grep "failed" | wc -l         # search compressed logs
tail -f pipeline.log | grep --line-buffered ERROR   # live error watch
find /data -name "*.csv" -mtime +30          # files older than 30 days
du -sh /data/* | sort -h                     # biggest folders last
```

15. Interview Q&A

Q. Is Linux an operating system?
Strictly, Linux is the kernel. A full OS is the kernel plus shell, utilities and packages, which is what a distribution like Ubuntu provides.

Q. Kernel vs shell?
The kernel manages hardware (processes, memory, devices). The shell takes your commands and hands them to the kernel.

Q. What is the difference between > and >>?
> overwrites the file; >> appends to it.

Q. How do you combine commands?
With a pipe (|), which sends one command's output to the next, e.g. `cut -d, -f2 f.csv | sort | uniq -c`.

Q. grep vs find?
grep searches *inside* files for text; find searches for *files* by name, size, time and so on.

Q. How do you count rows in a large file?
`wc -l file`. It streams the file, so it handles huge files without loading them into memory.

Q. How do you remove duplicate lines?
`sort file | uniq` or `sort -u file`. uniq only removes adjacent duplicates, so sort first.

Q. What does chmod 755 mean?
Owner can read, write and execute (7); group and others can read and execute (5).

Q. Explain the permission bits.
r=4, w=2, x=1, set separately for owner, group and others. 644 is typical for data files, 755 for scripts and directories.

Q. What does umask 022 do?
It removes write permission for group and others from new files, giving 644 for files and 755 for directories.

Q. How do you find which process uses a port or file?
`lsof -i :PORT` or `ss -tulpn` for ports; `lsof file` for files.

Q. df vs du?
df shows free/used space per file system; du shows how much space a given directory uses.

Q. How do you check memory and CPU?
free -h and top/htop for memory and CPU; vmstat, iostat and mpstat for deeper stats.

Q. kill vs kill -9?
kill sends SIGTERM, a polite request that lets the process clean up. kill -9 sends SIGKILL, which cannot be caught. Use it last.

Q. How do you run a job in the background and keep it alive after logout?
`nohup cmd &`. Check with jobs or ps.

Q. How do you schedule a recurring job?
Use cron via crontab -e. Format: minute hour day month weekday command. Redirect output to a log file.

Q. How would you monitor a log in real time?
`tail -f app.log`, optionally piped to grep to filter.

Q. What is a hard link vs soft link?
A hard link is another name for the same data (inode); a soft (symbolic) link is a shortcut pointing to a path and breaks if the target is removed. Create with `ln` / `ln -s`.

Q. tar -czf vs tar -xzf?
-c creates and -x extracts; z uses gzip; f names the file.

Q. apt vs yum?
Both are package managers: apt for Debian/Ubuntu, yum (and newer dnf) for Red Hat, CentOS, Fedora and Amazon Linux.

Q. curl vs wget?
Both fetch from URLs. wget is mainly for downloading files; curl is more flexible for calling APIs and sending requests.

Q. scp vs rsync?
scp copies everything. rsync copies only differences, can resume, and is better for large or repeated transfers.

Q. What is stdin, stdout, stderr?
Streams numbered 0, 1 and 2: input, normal output and error output. Redirect with <, > and 2>.

Q. How do you check disk is nearly full and find the cause?
df -h to see which file system is full, then du -sh /path/* | sort -h to find the biggest folders.

Q. How do you switch users and run as root?
`sudo su -` for root, `su - user` for another user, or prefix a single command with sudo.

16. Last-minute revision checklist

- Linux = kernel; OS = kernel + shell + utilities. Users: root, normal, system.
- Navigate: pwd, ls -lah, cd, mkdir -p, cp, mv, rm, find.
- Read: cat, less, head, tail -f. Edit: vi (i, Esc, :wq) or nano.
- > overwrite, >> append, | pipe, 2>&1 merge errors.
- Text power tools: grep, sed, awk, cut, sort, uniq -c, wc -l, tr, tee.
- Permissions: rwx = 421; chmod 755/644; chown user:group; umask 022.
- Processes: ps aux, top/htop, kill, nohup, &, jobs, fg, bg.
- Disk and memory: df -h, du -sh, lsblk, mount, free -h, vmstat, iostat.
- Archives: tar -czf / -xzf, gzip, zip, zcat.
- Network: ping, ip addr, ss, curl, wget, ssh, scp, rsync.
- Packages: apt (Ubuntu) vs yum/dnf (Red Hat family). Services: systemctl start/stop/status/enable.
- Scheduling: cron (m h dom mon dow cmd), crontab -e / -l.